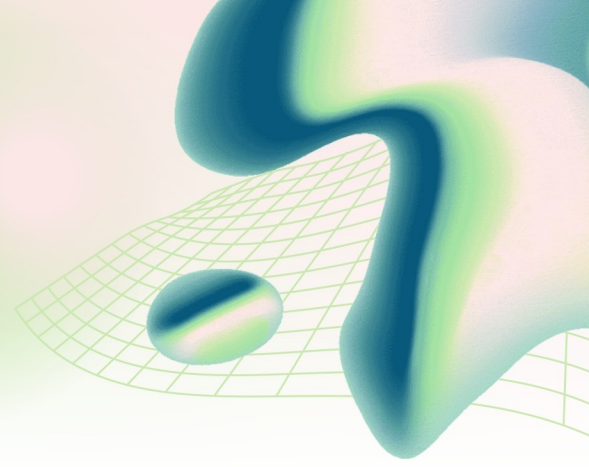


HABITLY

Sistema de Gestión Inmobiliaria en Java



Anexo 01 Protocolo de Rutas de Sistema y Almacenamiento Oculto

Versión 1.0.1A

Documento de evolución funcional y técnica

Autor: DevNaranjo

Año: 2026

Índice

1. Introducción.....	4
2. Problema Detectado.....	5
2.1 Riesgos principales del enfoque actual.....	5
3. Objetivo del Anexo.....	6
4. Principio de Diseño Aplicado.....	7
4.1 Código de aplicación.....	7
4.2 Datos de aplicación.....	7
5. Rutas Recomendadas por Sistema Operativo.....	8
5.1 Windows.....	8
Ventajas en Windows.....	8
5.2 Linux / Unix.....	8
Ventajas en Linux.....	9
6. Justificación Técnica de Seguridad y Protección.....	10
6.1 Beneficios principales.....	10
6.1.1 Protección frente a borrados accidentales.....	10
6.1.2 Independencia del proyecto fuente.....	10
6.1.3 Mejor preparación para futuras funciones.....	11
7. Estructura Recomendada de Datos de Aplicación.....	12
7.1 Explicación de esta estructura.....	12
8. Lógica Técnica Requerida.....	14
9. Flujo Lógico de Funcionamiento.....	15
9.1 Paso 1.....	15
9.2 Paso 2.....	15
9.3 Paso 3.....	15
9.4 Paso 4.....	15
9.5 Paso 5.....	15
9.6 Paso 6.....	15
9.7 Paso 7.....	15
10. Lógica Técnica Esperada (Conceptual).....	16
10.1 Comportamiento esperado.....	16
11. Propuesta de Responsabilidad Técnica por Clase.....	17
11.1 Opción recomendada.....	17
12. Requisitos Funcionales Asociados.....	18
12.1 RF-I.1.....	18
12.2 RF-I.2.....	18
12.3 RF-I.3.....	18
12.4 RF-I.4.....	18
12.5 RF-I.5.....	18
13. Requisitos No Funcionales Asociados.....	19
13.1 RNF-I.1 — Robustez.....	19
13.2 RNF-I.2 — Portabilidad.....	19
13.3 RNF-I.3 — Mantenibilidad.....	19

Anexo 01 – v1.0.01A

13.4 RNF-I.4 — Seguridad Operativa.....	19
14. Consideraciones de Migración desde la BETA.....	20
14.1 Recomendación de migración.....	20
14.1.1 Beneficios.....	20
15. Riesgos si No se Implementa esta Mejora.....	21
16. Conclusión.....	22

1. INTRODUCCIÓN

En la versión **v1.0.1-BETA** de Habitly se implementó un sistema funcional de persistencia basado en **serialización de objetos Java**, permitiendo guardar y recuperar el inventario de viviendas mediante un archivo binario con extensión `.dat`.

Actualmente, dicho archivo se almacena de forma visible y directa dentro del entorno de trabajo del proyecto, lo que supone una solución válida para la etapa beta, pero **insuficiente desde una perspectiva de seguridad, mantenimiento y experiencia de usuario**.

Por este motivo, de cara a la evolución técnica de **Habitly v1.0.1A**, se plantea la incorporación de un **protocolo de almacenamiento oculto y desacoplado del proyecto fuente**, trasladando el archivo `inventario_habitly.dat` a una **ruta de sistema específica del usuario**.

Este anexo define la justificación, el diseño conceptual y la lógica técnica que deberá seguir esta mejora.

2. PROBLEMA DETECTADO

Durante la fase beta, el archivo de persistencia se encuentra en una ubicación fácilmente accesible y potencialmente expuesta, normalmente:

`/inventario_habitly.dat`

o dentro del directorio del proyecto Java.

Aunque esto facilita el desarrollo inicial, también introduce varios riesgos técnicos y funcionales.

2.1 Riesgos principales del enfoque actual

- **Borrado accidental del archivo**
 - El usuario o desarrollador puede eliminar el archivo sin darse cuenta al limpiar carpetas del proyecto.
- **Pérdida de datos por reorganización manual**
 - Si se mueve la carpeta del proyecto, el archivo puede quedar fuera de contexto o desaparecer.
- **Exposición innecesaria de datos**
 - El archivo queda demasiado visible dentro del sistema de archivos.
- **Dependencia del entorno de desarrollo**
 - La persistencia queda ligada a la estructura de IntelliJ o del proyecto fuente, en lugar de comportarse como una aplicación independiente.
- **Mala escalabilidad futura**
 - Si más adelante se añaden configuraciones, caché, logs o preferencias, no habrá una ruta organizada para alojarlos.

En resumen, **la ubicación actual del archivo es funcional para pruebas, pero no adecuada para una aplicación en crecimiento.**

3. OBJETIVO DEL ANEXO

El objetivo de esta mejora es definir una política clara para que Habitly almacene sus datos persistentes en una **ruta segura, estable y específica del usuario del sistema operativo**, evitando que la información crítica dependa directamente del directorio del proyecto.

Esto permitirá que Habitly:

- sea más robusto ante errores humanos,
- desacople su almacenamiento del entorno de desarrollo,
- se comporte más como una aplicación real,
- y prepare una base más profesional para futuras versiones.

4. PRINCIPIO DE DISEÑO APLICADO

La mejora propuesta se basa en un principio fundamental de diseño de software:

Los datos de aplicación no deben depender de la carpeta del código fuente.

El código fuente puede:

- moverse,
- refactorizarse,
- clonarse,
- borrarse,
- exportarse,
- o incluso reconstruirse desde GitHub.

Sin embargo, **los datos del usuario deben mantenerse en una ubicación más estable y persistente**, propia del sistema operativo.

Por tanto, Habitly debe empezar a separar:

4.1 Código de aplicación

Archivos .java, configuración de IntelliJ, estructura Git, etc.

4.2 Datos de aplicación

Inventario, configuración interna, historial futuro, preferencias, etc.

Esta separación mejora la mantenibilidad y profesionaliza la arquitectura del proyecto.

5. RUTAS RECOMENDADAS POR SISTEMA OPERATIVO

Habitly deberá guardar sus datos en una carpeta específica de usuario, según el sistema operativo detectado.

5.1 Windows

En sistemas Windows, la ruta recomendada es:

`%AppData%\Habitly\`

Ejemplo real:

`C:\Users\NombreUsuario\AppData\Roaming\Habitly\`

Archivo final:

`C:\Users\NombreUsuario\AppData\Roaming\Habitly\inventario_habitly.dat`

Ventajas en Windows

- Ruta estándar para datos de aplicación.
- Separación del escritorio o carpetas visibles.
- Menor probabilidad de borrado accidental.
- Mayor consistencia con el comportamiento esperado de una aplicación instalada.

5.2 Linux / Unix

En sistemas Linux o Unix-like, la ruta recomendada es:

`~/.habitly/`

Ejemplo real:

`/home/usuario/.habitly/`

Archivo final:

`/home/usuario/.habitly/inventario_habitly.dat`

Ventajas en Linux

- Uso de carpeta oculta estándar.
- Aislamiento del almacenamiento del usuario.
- Compatibilidad con convenciones típicas del sistema.

6. JUSTIFICACIÓN TÉCNICA DE SEGURIDAD Y PROTECCIÓN

Mover el archivo `inventario_habitley.dat` fuera de la raíz del proyecto no implica un cifrado por sí mismo, pero **sí mejora significativamente la seguridad operativa y la resiliencia del sistema.**

6.1 Beneficios principales

6.1.1 Protección frente a borrados accidentales

Cuando el archivo se encuentra en una carpeta de proyecto visible, el riesgo de borrado es alto por acciones como:

- limpiar archivos “innecesarios”
- eliminar carpetas antiguas
- reorganizar documentación
- mover backups o versiones beta

Al situar el archivo en una ruta de sistema, se reduce drásticamente este riesgo.

6.1.2 Independencia del proyecto fuente

El inventario de viviendas no debería perderse si el usuario:

- cambia de IDE,
- clona de nuevo el repositorio,
- borra una versión antigua,
- o reorganiza el código fuente.

Con una ruta de usuario persistente, Habitly deja de depender del directorio de desarrollo.

6.1.3 Mejor preparación para futuras funciones

Esta decisión también prepara el sistema para alojar más adelante otros elementos como:

- archivos de configuración
- logs internos
- caché temporal
- backups automáticos
- historial de operaciones
- preferencias de usuario

Por tanto, no se trata solo de mover un .dat, sino de empezar a construir una **estructura de almacenamiento interna escalable**.

7. ESTRUCTURA RECOMENDADA DE DATOS DE APLICACIÓN

A medio plazo, Habitly debería trabajar con una estructura interna similar a esta:

```
Habitly/  
├── inventario_habitly.dat  
├── config.properties  
├── logs/  
│   └── habitly.log  
├── cache/  
│   └── temp_cache.dat  
└── backups/  
    └── inventario_backup.dat
```

7.1 Explicación de esta estructura

- inventario_habitly.dat
 - Archivo principal de persistencia del inventario.
- config.properties
 - Configuración futura del sistema.
- logs/
 - Registros técnicos para depuración o auditoría.
- cache/
 - Archivos temporales o estructuras de apoyo.
- backups/

Anexo 01 – v1.0.01A

- Copias de seguridad automáticas o manuales.

En **v1.0.1A** no es necesario implementar todo esto todavía, pero sí es importante que la ruta elegida ya permita crecer en esa dirección.

8. LÓGICA TÉCNICA REQUERIDA

Habitly deberá implementar una lógica de inicialización que garantice que la carpeta de datos exista antes de intentar guardar o leer el archivo.

Esto implica tres responsabilidades técnicas:

- A) Detectar la ruta correcta según el sistema
- B) Verificar si el directorio existe
- C) Crearlo automáticamente si no existe

9. FLUJO LÓGICO DE FUNCIONAMIENTO

El comportamiento esperado del sistema debe ser el siguiente:

9.1 Paso 1

Al iniciar Habitly, el programa determina la ruta de almacenamiento de datos según el sistema operativo.

9.2 Paso 2

El sistema verifica si la carpeta base de Habitly ya existe.

9.3 Paso 3

Si no existe, el programa debe crearla automáticamente.

9.4 Paso 4

Una vez asegurado el directorio, el sistema busca el archivo:

`inventario_habitly.dat`

9.5 Paso 5

Si el archivo existe:

- se intenta cargar el inventario serializado.

9.6 Paso 6

Si el archivo no existe:

- se crea una estructura de inventario vacía en memoria.

9.7 Paso 7

Cada vez que se guarde el inventario:

- se utilizará siempre esta ruta fija de aplicación.

10. LÓGICA TÉCNICA ESPERADA (CONCEPTUAL)

A nivel de diseño, el sistema deberá incorporar una utilidad o módulo responsable de construir la ruta persistente.

10.1 Comportamiento esperado

- Leer la propiedad del sistema que indica el sistema operativo o el directorio del usuario.
- Resolver la ruta adecuada para Habitly.
- Crear la carpeta si no existe.
- Devolver una referencia reutilizable al archivo de inventario.

11. PROPUESTA DE RESPONSABILIDAD TÉCNICA POR CLASE

Para mantener una arquitectura limpia, esta lógica **no debería quedar incrustada directamente en el main.**

Lo recomendable es separar responsabilidades.

11.1 Opción recomendada

Crear una futura clase auxiliar, por ejemplo:

`RutaSistema`

o

`GestorPersistencia`

Posibles responsabilidades de esta clase

- obtener la ruta de almacenamiento
- construir la carpeta de datos
- devolver la ruta del archivo .dat
- centralizar la persistencia futura

Esto ayuda a que `GestionInmobiliaria` no acumule demasiadas tareas técnicas ajenas al flujo de negocio.

12. REQUISITOS FUNCIONALES ASOCIADOS

Para considerar esta mejora correctamente implementada en futuras versiones, el sistema deberá cumplir los siguientes requisitos:

12.1 RF-I.1

Habitly deberá guardar el archivo `inventario_habitly.dat` fuera de la carpeta raíz del proyecto.

12.2 RF-I.2

La ruta utilizada deberá ser específica del usuario del sistema operativo.

12.3 RF-I.3

El sistema deberá crear automáticamente el directorio de datos si no existe.

12.4 RF-I.4

La carga y guardado del inventario deberán funcionar de forma transparente para el usuario.

12.5 RF-I.5

La migración no deberá romper el comportamiento funcional ya existente de persistencia.

13. REQUISITOS NO FUNCIONALES ASOCIADOS

13.1 RNF-I.1 — Robustez

La aplicación no deberá fallar si la carpeta de datos aún no existe.

13.2 RNF-I.2 — Portabilidad

La solución deberá ser adaptable a distintos sistemas operativos.

13.3 RNF-I.3 — Mantenibilidad

La lógica de rutas deberá estar desacoplada del flujo principal del menú.

13.4 RNF-I.4 — Seguridad Operativa

El archivo no deberá permanecer en una ubicación expuesta o fácilmente manipulable por error.

14. CONSIDERACIONES DE MIGRACIÓN DESDE LA BETA

Dado que la versión beta guardaba el archivo en una ubicación más simple, será necesario contemplar una transición ordenada.

14.1 Recomendación de migración

En una primera implementación de esta mejora, Habitly podrá:

- comprobar si existe un archivo antiguo en la ruta vieja,
- y si existe, ofrecer migrarlo automáticamente a la nueva ruta.

14.1.1 Beneficios

- evita pérdida de datos de usuarios beta
- mantiene compatibilidad con el trabajo ya realizado
- facilita una transición limpia entre versiones

Esta funcionalidad no es estrictamente obligatoria para la primera iteración, pero sí muy recomendable.

15. RIESGOS SI NO SE IMPLEMENTA ESTA MEJORA

Si Habitty continúa guardando sus datos en la raíz del proyecto o en rutas visibles de desarrollo, se mantendrán los siguientes riesgos:

- pérdida accidental del inventario
- dependencia excesiva del IDE o del entorno local
- dificultad para escalar el almacenamiento
- mala práctica de separación entre aplicación y datos
- menor profesionalidad del sistema

Por tanto, esta mejora no debe considerarse “cosmética”, sino una **mejora estructural importante**.

16. CONCLUSIÓN

La migración del archivo `inventario_habitly.dat` a una ruta de almacenamiento específica del sistema operativo representa una evolución técnica lógica y necesaria para Habitly.

Aunque durante la fase beta resultó razonable mantener una persistencia sencilla dentro del entorno de desarrollo, la madurez creciente del proyecto exige una solución más robusta y coherente con el comportamiento esperado de una aplicación real.

La adopción de una ruta oculta o desacoplada del proyecto permitirá:

- proteger mejor la información,
- reducir el riesgo de pérdida accidental,
- profesionalizar la persistencia,
- y preparar el terreno para futuras mejoras de seguridad y mantenimiento.

Este anexo servirá como referencia de implementación para la evolución de Habitly en versiones posteriores, especialmente en el contexto de las mejoras previstas para la línea **v1.1**.